Rapport de Projet

Méthodes de Conception : Librairie Cartes & Blackjack

Nazar ULAN
Université de Caen Normandie
Licence 3 Informatique

Décembre 2025

Table des matières

1 Introduction **3**

2 Architecture Logicielle **3**
2.1 Organisation en Modules 3
2.2 Architecture MVC (Modèle-Vue-Contrôleur) 3

3 Patrons de Conception (Design Patterns) **3**
3.1 Pattern Observer (Observateur) 3
3.2 Pattern Strategy (Stratégie) 4
3.3 Pattern Factory (Fabrique) 4
3.4 Pattern Flyweight (Poids-mouche) - Cache d'images 4

4 Choix Techniques et Algorithmes **5**
4.1 Gestion de la valeur de l'As 5
4.2 Automatisation avec Apache Ant 5

5 Tests et Validation **5**

6 Conclusion **6**

1 Introduction

Ce projet s'inscrit dans le cadre de l'unité d'enseignement "Méthodes de Conception". L'objectif principal était de concevoir et développer une application de jeu de cartes, le **Blackjack**, en respectant les principes de la conception orientée objet et en assurant une qualité logicielle professionnelle.

Le projet a été structuré en deux modules distincts pour garantir la réutilisabilité du code :

- Une librairie générique `cartes`, capable de gérer des paquets et des cartes indépendamment des règles du jeu.
- Une application `blackjack`, qui implémente la logique métier et l'interface graphique du jeu en utilisant la librairie.

Ce rapport détaille l'architecture logicielle retenue, les patrons de conception (Design Patterns) mis en œuvre, ainsi que les choix techniques garantissant la robustesse et la maintenabilité du projet.

2 Architecture Logicielle

2.1 Organisation en Modules

Afin de respecter le principe de séparation des responsabilités, nous avons isolé la gestion des cartes dans une archive JAR autonome (`cartes.jar`). Le jeu de Blackjack dépend de cette librairie mais la librairie ne connaît pas le jeu.

2.2 Architecture MVC (Modèle-Vue-Contrôleur)

Nous avons strictement appliqué le patron architectural MVC pour découpler les données de leur représentation graphique.

- **Modèle :** Les classes `BlackjackGame`, `Player` et `Paquet` encapsulent l'état du jeu. Elles n'ont aucune dépendance vers la bibliothèque Swing.
- **Vue :** Les classes `SwingBlackjack`, `VuePaquetVisible` et `VueDealer` sont chargées de l'affichage. Elles implémentent l'interface `EcouteurModele` pour réagir aux changements.
- **Contrôleur :** Les interactions utilisateur (boutons "Tirer", "Rester") déclenchent des méthodes du modèle, qui à son tour notifie la vue.

3 Patrons de Conception (Design Patterns)

L'utilisation de design patterns a permis de résoudre des problèmes récurrents de conception de manière élégante.

3.1 Pattern Observer (Observateur)

Pour lier le Modèle et la Vue sans couplage fort, nous avons implémenté le pattern Observer.

- `ModeleEcoutable` (Sujet) : Définit les méthodes d'abonnement.
- `EcouteurModele` (Observateur) : Définit la méthode de mise à jour.

Listing 1 – Implémentation de la notification

```
protected void fireChangement() {
    for (EcouteurModele e : ecouteurs) {
        e.modeleMisAJour(this);
    }
}
```

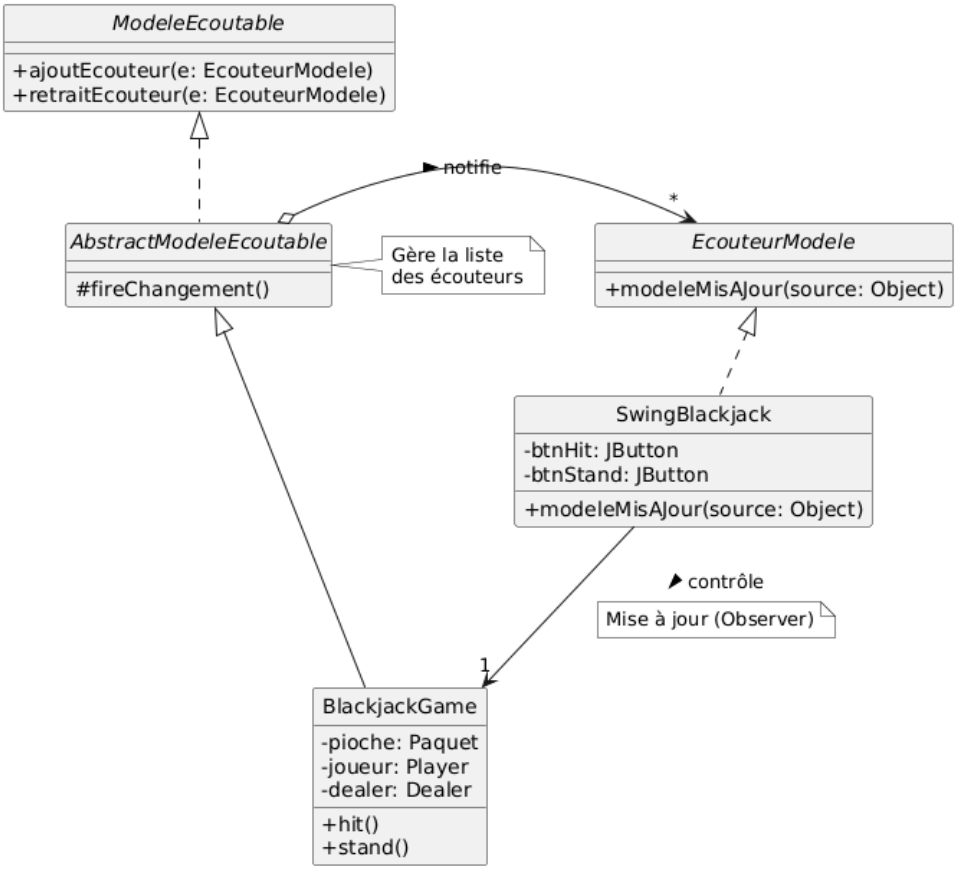

FIGURE 1 – Diagramme de classes simplifié de l'architecture MVC

Note sur l'implémentation Swing :
Il est important de souligner que le pattern Observer est omniprésent dans le MVC. En plus de notre implémentation (Modèle vers Vue), Java Swing utilise ce pattern pour gérer les interactions : les contrôleurs "observent" les clics souris de la Vue via l'interface `MouseListener`.

3.2 Pattern Strategy (Stratégie)

Le comportement du Dealer (Croupier) ne doit pas être figé. Nous avons utilisé le pattern Strategy pour encapsuler l'algorithme de décision de l'IA. Cela permet de changer la difficulté du Dealer dynamiquement.

— **Interface :** `StrategieJeu`
— **Concrétions :** `StrategieDealerBase` (règle du casino) et `StrategiePrudente`.

3.3 Pattern Factory (Fabrique)

La création d'un paquet de 52 cartes est complexe (boucles imbriquées, gestion des figures). La classe `FabriquePaquet` centralise cette logique via la méthode statique `creerPaquet52()`. Cela simplifie le code client.

3.4 Pattern Flyweight (Poids-mouche) - Cache d'images

Pour optimiser les performances de l'affichage Swing, la classe `VuePaquetVisible` utilise un cache statique (`HashMap`). Les images des cartes (PNG) ne sont chargées qu'une seule fois en mémoire, même si elles sont affichées plusieurs fois à l'écran.

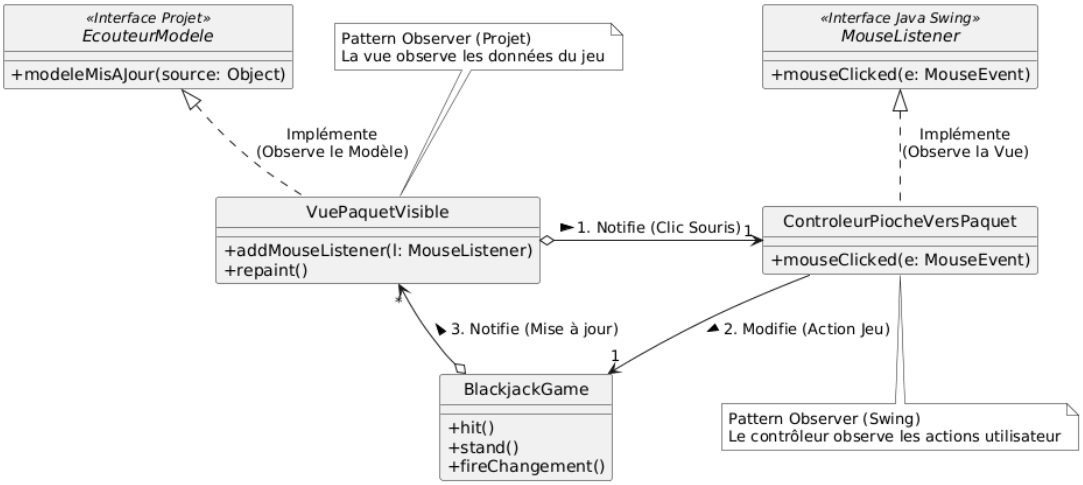

FIGURE 2 – Double application de l'Observer : Modèle-Vue et Vue-Contrôleur

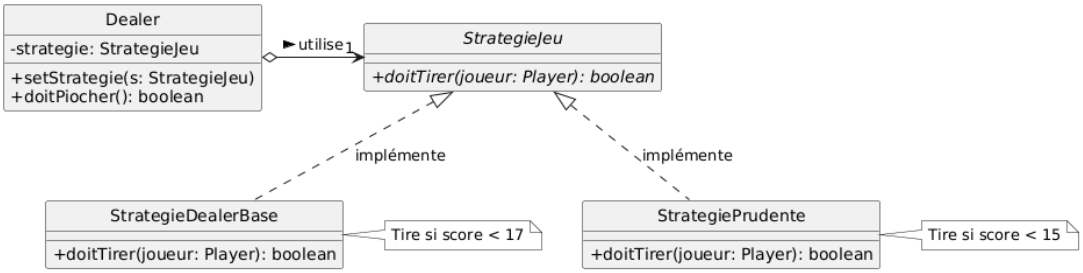

FIGURE 3 – Mise en œuvre du pattern Strategy pour l'IA du Dealer

4 Choix Techniques et Algorithmes

4.1 Gestion de la valeur de l'As

Un algorithme spécifique a été développé dans la classe `Player` pour calculer le score optimal. L'As vaut 11 points par défaut, mais passe à 1 point si le total dépasse 21.

```
// Ajustement dynamique des As
while (score > 21 && as > 0) {
    score -= 10;
    as--;
}
```

4.2 Automatisation avec Apache Ant

Le projet est livré avec un fichier `build.xml` complet qui automatise :

- La compilation des deux modules.
- La génération de la Javadoc (`ant javadoc`).
- La création des fichiers JAR exécutables et l'inclusion des images (`ant dist`).
- L'exécution des tests unitaires (`ant test`).

5 Tests et Validation

La robustesse de la librairie `cartes` a été validée par une suite de tests unitaires (`TestJeuDeCartes.java`) vérifiant :

1. La création conforme d'un paquet de 52 cartes.
2. L'efficacité du mélange (comparaison avant/après).
3. La mécanique de pioche et de retrait de carte.

Le jeu complet a été testé fonctionnellement pour valider les conditions de victoire, d'égalité et de défaite ("Bust").

6 Conclusion

Ce projet a permis de mettre en pratique une architecture logicielle rigoureuse. L'utilisation du MVC et des patrons de conception assure un code lisible, maintenable et évolutif. Le jeu est fonctionnel et l'environnement de développement est entièrement automatisé.

Des évolutions futures, comme l'ajout de la règle du "Split" ou un mode multijoueur réseau, seraient grandement facilitées par la séparation modulaire actuelle.